Nima

TECHNICAL PLAN · V3

- Overview
- Stack
- Virtual try-on
- Responsive web
- Data model
- Realtime
- VPS & ops
- Lessons from old code
- What "free" means
- Architecture
- Other AI
- Monorepo
- Key flows
- Seller analytics
- Roadmap
- Open questions

UPDATED 21 SEP 2026

Nima, built on open source

AI stylist, virtual try-on, social fashion feed and marketplace, Kenya-first (KSh, M-Pesa, Pick Up Mtaani). One Expo app for iOS, Android and a responsive web. Everything is open source and self-hosted on one Contabo VPS, except try-on images, which use Google Gemini the way the original app did.

API Fastify 5 + TS | DB Postgres 17 + Prisma | Auth Better Auth | Realtime WebSockets | App Expo (iOS · Android · Web) | Try-on Google Gemini

01 What we're building

From the brief and the 11 Canva screens.

- **Top navigation:** `Shop` · `For You` · `My Store`. Buyers and sellers use one account.
- **For You:** a full-bleed look card with Love / Save / Comments and a **"Nima Says"** AI caption.
- **Shop:** a masonry grid. **Auth:** a bottom sheet (Apple / Google / email) that appears only on gated actions.
- **Share sheet:** friend avatars, search, a group icon and Send.
- **Profile:** credits pill, style tags, highlight circles, and tabs: Looks · Lookbooks · Wardrobe · Recreations · Orders.

Cream #FAF8F3 **Deep brown** #4A2A12 **Terracotta** #B8645A **Type** Cormorant-style serif + DM Sans

Credits (decided)

Every AI action spends credits from a **credit ledger**. Free users get a weekly grant. Packs are bought with M-Pesa on Android and web, and with Apple In-App Purchase on iPhone (Apple rejected M-Pesa for digital goods in the old app). Costs per action are settings, not code. Failed jobs refund automatically.

STARTING NUMBERS (FROM THE OLD APP)	VALUE
Free weekly credits	5 (the brief suggests ~20; raise once GPU cost is known)
Credit packs	10 / 20 / 50 / 100 credits for KES 500 / 1,000 / 2,500 / 5,000 (KES 50 per credit)
Try-on, quick camera try-on, create a look	1 credit each
3 looks from Ask Nima	3 credits
Onboarding looks	Free
Recreate, retry, wardrobe processing	Charged. The old app forgot to charge these

02 What "free" really means

The software is free. These things still cost money. The App Store and Google Play accounts already exist.

UNAVOIDABLE	WHY	HOW WE MINIMISE IT
Contabo VPS	Something has to run the server	One VPS with lots of RAM, because the LLM runs in RAM
Domain name	HTTPS, email and app links	About $10/yr
Apple's 15–30% cut	Credits bought on iPhone must use In-App Purchase	iOS only; Android and web use M-Pesa
Fingo Pay fees	Charged per transaction, taken from sales	Priced into the platform fee
Pick Up Mtaani fees	Charged per parcel	The buyer pays at checkout
Google Gemini API	Try-on images, paid per image (≈ $0.05–0.13 each, est.)	Credits cover it; cheaper Flash model by default
SMS login codes	Every SMS costs money	Skipped Email codes, magic links, Google/Apple instead

Sign in with Apple is still required when Google login is offered. **Not used:** OpenAI, Claude API, Photoroom, Cloudflare R2, Resend, PostHog cloud, Tailscale.

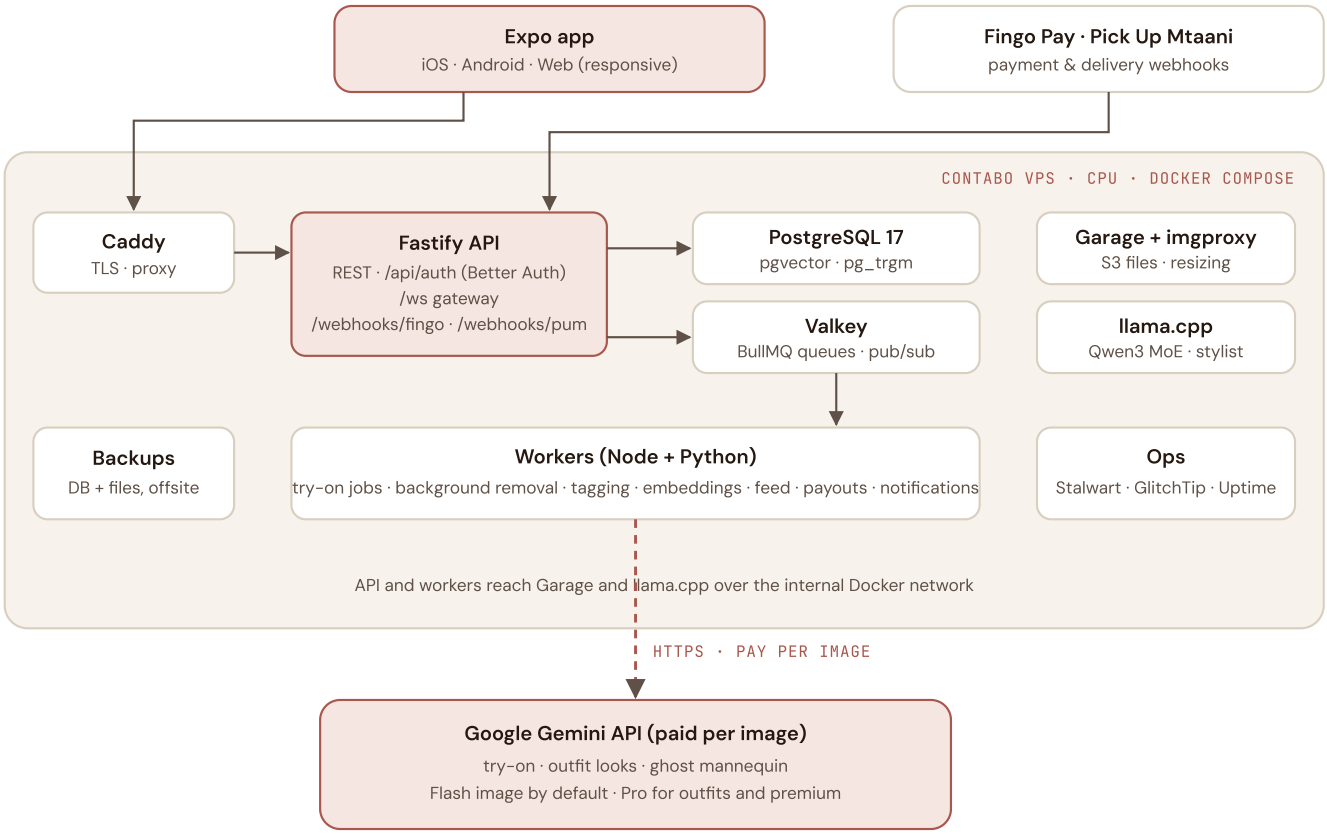
03 Stack

Open source and self-hosted, except Gemini for try-on images.

CONCERN	CHOICE	NOTES
API	Fastify 5 + TypeScript (ESM)	MIT
Contracts	Zod + `fastify-type-provider-zod` → OpenAPI → typed client	One source of truth for the app and webhooks
Database	PostgreSQL 17 + Prisma, with `pgvector`, `pg_trgm`, `citext`	Vectors via `Unsupported("vector(n)")` + raw SQL
Auth	Better Auth 1.7: guests, Google, Apple, email OTP, magic link, passkeys, Expo	A guest's data moves to their real account on sign-up. No SMS
Realtime	`@fastify/websocket` + Valkey pub/sub	Ready for more API nodes later
Jobs	BullMQ on Valkey	All slow work is queued
File storage	Garage or SeaweedFS (S3-compatible)	Not MinIO, which was archived in 2026
Images	imgproxy behind Caddy	Resizing, WebP/AVIF
Search	Postgres full-text + `pg_trgm`, Meilisearch later	No extra service on day one
Email	Stalwart or Postal	SPF/DKIM/DMARC; needs IP warm-up for Gmail
Push	Expo push service	Free, no per-message fee
Monitoring	GlitchTip + Uptime Kuma	Sentry-compatible, lightweight
Try-on images	Google Gemini API	The only paid AI service
Deploy	Docker Compose + Caddy	Automatic HTTPS
App	Expo SDK + Expo Router + NativeWind	iOS, Android and responsive web from one codebase
Monorepo	Turborepo + pnpm	Shared types, DB and UI

04 Architecture

Everything runs on one VPS. Workers send try-on images to Google Gemini and push the results back over WebSocket.



- The API never does slow work inline. It queues a job, returns a `jobId`, and the result arrives over WebSocket.
- Postgres and Valkey listen only on the internal Docker network, with passwords. Only ports 80/443 and SSH are public.
- Every AI model or outside service sits behind an interface (`TryOnProvider`, `LLMProvider`, `PaymentProvider`, `LogisticsProvider`…), so we can swap it without touching the rest of the code.

05 Virtual try-on: Gemini, done better

We keep the original app's approach and fix what made it slow, costly and leaky.

How the original app did it

1

Request

The user taps try-on. Credits are taken and a pending job is created.

2

Download

The server fetches the user's photo and up to 5 item photos, at full size.

3

GPT-4o

Writes a "photography director" prompt: keep face and body, show every item.

4

Gemini

`gemini-3-pro-image-preview` dresses the person, falling back to Flash when busy, with one retry.

5

Save

The image is stored and the app updates.

The same recipe powered item, outfit, camera Quick Try-On, recreate, seller links, the partner API, wardrobe clean-up and ghost mannequin. **Why keep it:** Gemini handles several items and accessories at once, which dedicated try-on models don't. It's proven on Kenyan outfits and needs no GPU of our own.

What we do better

CHANGE	WHY
Drop the GPT-4o step; build the prompt from a template plus stored item data	One less paid service, 2–5 s faster, no OpenAI account
Cheaper Gemini Flash image by default; Pro for full outfits and premium	Pro costs roughly 2× Flash. The week-1 test decides where each is good enough
Resize photos to ~1024 px before sending	Lower cost, faster. The old app sent full-size photos
"Change the top" sends the previous result as the base image	The person stays consistent across edits
Free photo check before charging	No credits wasted on unusable photos
Cache: same photo + same items reuses the result	No double charge on re-opens and shares
One charge-then-queue function for every image job, refund on failure	Fixes the old free recreate, retry and chat-look leaks
No user photo means no generation	The old fallback showed a random person
Provider switch built in	Can A/B-test FASHN (dedicated try-on, ≈ $0.075, sharper prints and logos) later
Live result over WebSocket	"Nima is dressing you…", then the image appears. No polling

Cost maths

Estimates. Confirm on Google's pricing page before launch.

ITEM	PER IMAGE	NOTES
Gemini Pro image	≈ $0.13	What the old app used by default
Gemini Flash image	≈ $0.05–0.07	Our default
One credit sold	KES 50 ≈ $0.39	iOS nets 15–30% less after Apple's cut

Free credits are the real cost

Paid try-ons are profitable, since a credit covers even a Pro image about 3×. But 5 free try-ons a week costs ≈ $0.25–0.65 per active free user, so **1,000 active free users ≈ $1,100–2,800/month** in the worst case. That's why the free grant starts at 5, not 20. Watch it and tune it.

Privacy: user photos go to Google. Say so at upload and in the privacy policy, and use the paid API tier.

How a try-on runs in the new app

1

Request

The user picks items and a photo; the app calls `POST /tryon`.

2

Check + charge

Photo check, then credits debited and job queued in one transaction.

3

Gemini

A VPS worker resizes the images, builds the prompt and calls Flash or Pro.

4

Deliver

Result stored privately; `tryon.done` over WebSocket. Refund on failure.

5

Notice

"Visualization only — check the seller's size guide."

Accessories (jewellery, sunglasses) use the upper-body photo in the same pipeline, labelled "beta" at first.

06 Other AI (open weights, self-hosted, free)

JOB	MODEL	LICENSE	RUNS ON
Ask Nima stylist	Qwen3 30B-A3B MoE via llama.cpp (fallback: Qwen3.5-4B / Gemma 4 E4B)	Apache-2.0	VPS CPU, est. 5–10 tokens/s
"Nima Says" captions, tags	Qwen3.5-4B / Gemma 4 E4B	Apache-2.0	VPS CPU, generated in batches
Wardrobe photo tagging	Qwen3-VL-4B or Florence-2	Apache-2.0 / MIT	VPS CPU, async
Recommendations (embeddings)	Marqo-FashionSigLIP → pgvector	Apache-2.0	VPS CPU
Background removal	BiRefNet (via rembg, selected explicitly)	MIT	VPS CPU, a few seconds per image
Ghost mannequin	Gemini image (paid), charged to seller credits	—	Gemini API
Upload moderation	Open NSFW classifier + Qwen3-VL check	Apache-2.0	VPS CPU

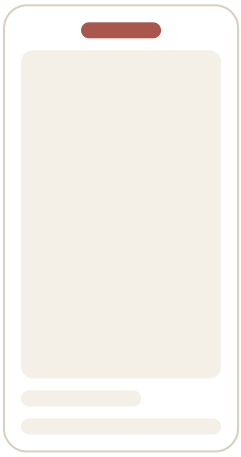
Making a CPU model feel fast

The model only works out what the user wants ("wedding, elegant, under KSh 5,000, size M"). Our code runs the search and returns real, in-stock products. The model then writes a short reply that streams word by word. Short prompts plus llama.cpp's prompt cache avoid long waits before the first word. If it still feels slow, a cheap hosted model (e.g. Gemini Flash text) can be swapped in.

VPS size: aim for 12+ vCPU and 48 GB+ RAM. The minimum is 8 vCPU / 32 GB with the smaller 4B model.

07 Responsive web, not a stretched phone

One codebase. The layout follows screen width, not device type, so a narrow browser window gets the phone layout.



Phone · < 768
Canva design: top pills, bottom tabs, slide-up sheets

Tablet · 768–1279
Icon rail, centred feed with a product strip, 3–4 column shop

Desktop · ≥ 1280
Full sidebar, feed in the middle, Ask Nima docked on the right, pop-up windows

WIDTH	NAVIGATION	FOR YOU	SHOP	SHEETS	ASK NIMA
< 768	Top pills + bottom tabs	Full-bleed card	2 columns	Bottom sheet	Full screen
768–1279	Left icon rail	Centred card + product strip	3–4 columns	Centred modal	Side panel
≥ 1280	Full left sidebar	Feed + right panel	4–6 columns + filters	Dialog	Docked right

- **Expo Router** shows `<Tabs>` on narrow screens and `<Sidebar>` on wide ones. The URLs stay identical.
- **Shared components:** `<Sheet>` (bottom sheet ↔ dialog), `<Masonry columns={{base:2, md:3, lg:4, xl:6}}>`, `<Container>`, `<SplitView>`.
- **Web polish:** hover states, keyboard focus, `Esc` closes dialogs, and links are real links (open in a new tab works).
- The **seller dashboard** is designed desktop-first and stacks into cards on phones.
- **Public pages** (product, look, store, profile) are pre-rendered, so Google indexes them and WhatsApp links show a preview.

08 Monorepo layout

```
nima/
├ apps/
│  ├ api/            Fastify (REST + WS + webhooks)
│  ├ worker/         Node BullMQ workers
│  ├ ml-cpu/         Python: BiRefNet, FashionSigLIP, Qwen-VL (VPS)
│  └ app/            Expo: iOS / Android / Web
├ packages/
│  ├ db/             Prisma schema, migrations, TypedSQL
│  ├ contracts/      Zod schemas + WebSocket event types
│  ├ api-client/     Generated typed client
│  ├ integrations/   gemini, fingo, pickupmtaani, llm, storage, mailer
│  ├ ai/             Stylist prompts, tools, context builder
│  ├ ui/             Responsive components + Canva tokens
│  └ config/         tsconfig, eslint, env schema
├ infra/            docker-compose, Caddyfile, backups
└ docs/
```

09 Data model (first cut)

IDENTITY

User, Session, Account (Better Auth; `isAnonymous` marks guests)

Profile handle, bio, links, avatar, private

StyleProfile all fields optional, autosaved per question, taste vector

UserPhoto full-body or upper-body, private

CATALOG

Store optional, one per user

Product size guide, return policy, embedding

ProductVariant size, colour, price, stock

ProductImage original + processed

LOOKS & SOCIAL

Look + LookItem try-ons private by default

WardrobeItem AI-tagged, embedding

Lookbook + Entry by day or occasion

Save, Like, NotForMe, Comment, Follow follower counts never public

Conversation, Message text, product, look, try-on, lookbook

AI

TryOnJob status, parent job for "change the top"

StylistThread + Message

CreditLedger weekly grant, try-on, refund

COMMERCE

Cart, CheckoutGroup, Order one order per store

Payment idempotency key, raw response

Shipment provider, pickup point, events

SellerLedgerEntry, Payout, Review, WebhookEvent

ANALYTICS & ADS

Event append-only, partitioned monthly

DailyStoreStats roll-ups

Promotion budget, targeting, dates

10 Key flows

Guest → account	Open app as a guest → browse, like and add to cart → a gated action opens the sheet: "Create an account to try this look on yourself." → the guest's data merges into the new account → the questionnaire is an optional card and never blocks
Ask Nima	The model works out intent and filters → tools run the search, wardrobe, style profile, matching items, look creation and try-on → only real, in-stock items → the reply streams
Wardrobe	Upload → BiRefNet cut-out → Qwen-VL tags → FashionSigLIP vector → appears in Wardrobe
For You	Candidates from taste match, follows, trending, seasonal sets and labelled promotions → weighted score, lowered by "not for me" and skips → the taste vector keeps learning from activity
Checkout	Cart split per store → choose a Pick Up Mtaani point → Fingo M-Pesa prompt → verified webhook marks the order paid and credits the seller's ledger → seller marks ready and a parcel is created → after collection and the return window, the seller is paid
Seller photos	Upload → BiRefNet white background (free) → optional Gemini ghost mannequin (seller credits) → seller picks an image

Legal check needed

Fingo can't split one payment between sellers, so Nima holds the money until delivery. That may raise Central Bank of Kenya licensing questions.

11 Realtime events

Connections are authenticated with the login session. Each user has a private channel. Offline users get a push notification instead.

`message.new` `message.read` `typing` `tryon.queued` `tryon.progress` `tryon.done` `tryon.failed` `stylist.delta` `stylist.done` `payment.status` `order.updated` `shipment.updated` `notification.new`

12 Seller analytics

Every interaction is logged against the store and rolled up daily. The headline feature is the funnel, which shows what shoppers do before they buy:

- Views
- Try-ons
- Added to cart
- Purchases

Premium sellers get grouped customer demographics and preferences. Groups have a minimum size, so no individual buyer can be identified. Follower counts are shown here only.

13 VPS & ops

- **Services:** caddy, api, worker, ml-cpu, llm (llama.cpp), postgres, valkey, garage, imgproxy, stalwart, glitchtip, uptime-kuma.
- **Backups:** nightly database dump plus continuous WAL archiving, and a Garage snapshot, copied offsite. Test a restore monthly.
- **Security:** firewall allowing only web and SSH; fail2ban; SSH keys only; non-root containers; secrets kept out of git; the database never public.

14 Roadmap

Weeks 1–2	**Foundations + try-on test** Monorepo, Docker Compose, Postgres, Valkey, Garage, Caddy, CI. Better Auth with guest, Google and email login. Expo shell with responsive layouts and Canva tokens. Try-on test: Gemini Flash vs Pro on real Kenyan outfit photos.
Weeks 3–5	**Discover & account** Stores and products, Shop grid, For You v1, looks, like/save/not-for-me, profile, autosaving questionnaire, photo upload, public pages.
Weeks 6–9	**AI core** Gemini try-on pipeline, credits, live updates. Ask Nima stylist with tools. Wardrobe tagging, embeddings, For You v2.
Weeks 10–11	**Social** Follows, Explore, DMs and sharing, Recreate Look, Lookbooks, push notifications. Seller try-on links.
Weeks 12–15	**Commerce** Seller onboarding, product management, background removal, cart, Fingo checkout, Pick Up Mtaani, orders, seller ledger and payouts, reviews.
Week 16+	**Growth** Seller analytics funnel, promotions, premium plan, admin and moderation, ghost mannequin, FASHN vs Gemini comparison.

15 Lessons from the old code

Four repos from github.com/zyraglobal7, cloned into `reference/`. Full audits are in `docs/legacy/`.

REPO	WHAT IT WAS	STATUS
Nima-Universe	Newest monorepo: Expo app + Next.js web (seller portal, admin, tailor portal, Wrapped) on one Convex backend (~43 tables)	37 commits, to Jul 2026
Nima-Convex	Next.js web app at shopnima.ai, the most-worked codebase (~100k lines)	187 commits, to Apr 2026
Nima-Native	Expo app on a copy of the same backend	31 commits, to Apr 2026
nima-demo	Fake shop showing the "Try on with Nima" widget for partner stores	Integration example only
docs	Untouched docs template	Nothing to reuse

How their try-on worked

No dedicated try-on model. GPT-4o wrote a "photography director" prompt, then Google Gemini (`gemini-3-pro-image-preview`) got the user's photo plus up to 5 item photos with the instruction to keep the face and body and dress them in every item. It retried once if the result was empty. The same recipe powered item, outfit, camera Quick Try-On, recreate, seller links and the partner API, plus wardrobe cleanup and ghost mannequin. All of it was paid. **We keep Gemini for images**, drop the GPT-4o step and OpenAI entirely, and add the improvements in section 05.

KEEP

- Database design blueprint for our Prisma schema
- AI prompts stylist, style profile, try-on, "Nima Says", photo check, wardrobe
- Outfit matcher occasion + formality scoring for For You v1
- Fingo M-Pesa code STK push + signed webhook
- Credit economics & seller tiers KES 0 / 5k / 15k / 30k per month
- Onboarding content, style images, UI flows

KEEP AS FEATURES

- Quick Try-On photograph a garment in a shop
- Onboarding looks 3 free looks after sign-up
- Seller try-on links
- Nima Connect partner shops embed try-on
- Nima Wrapped · tailor portal to confirm

DISCARD

- All Convex and WorkOS code
- Duplicate Next.js consumer app
- Legal texts wrong services, Delaware law; rewrite for Kenya
- Docs template, demo UI

Their mistakes, our rules

WHAT WENT WRONG	WHAT WE DO FROM DAY ONE
Any logged-in user could take over any account (the server trusted the email sent by the app)	Identity only comes from the login session; every webhook is signature-checked
Admin catalogue and admin AI had no login check	Role guards on every route, and tests that fail if one is missing
Profile lookups leaked email, phone and address	Separate public and private responses; private fields stripped
Any file could be deleted by its ID	Ownership checked before every file action
Negative cart quantities lowered the total	Strict validation; totals always computed on the server
Recreate, retry and chat looks generated images for free	One charge-then-queue function for every AI job, refund on failure
Rate limiter written but never switched on	Rate limits on all routes, tighter on AI and login
Deleting an account left photos in storage	Deletion removes database rows and files (Kenya Data Protection Act)
Blocking a user did nothing	Blocks enforced in feed, DMs, profiles and search
Money mixed cents and whole shillings	All money stored as integer cents
A try-on fallback dropped the user's photo and showed a random person	No user photo means no generation
No guest mode, empty Explore, look-only DMs, no courier	Guest accounts, Explore, text DMs, Pick Up Mtaani
iPad got a phone layout; Apple flagged it	Responsive layouts
No tests, lint or CI	Tests, lint and type checks in CI from week 1

16 Open questions

1. **Brand colours:** the Canva design (brown `#4A2A12` / cream / terracotta) or the old app (burgundy `#5C2A33` / ivory / rose gold)?
2. **Existing users:** shopnima.ai and the old backend look live. Move their users, products and looks into the new database, or start fresh? Keep the shopnima.ai domain?
3. **Contabo plan specs** (vCPU / RAM / disk). These decide which chat model size we can run.
4. **Fingo:** is there a card API? Can they split settlement so we avoid holding funds? What are the fees?
5. **Pick Up Mtaani API key:** request it via their support email; it needs a registered business.
6. **Platform fee** percentage, and the return window before sellers are paid.
7. **Old features:** keep Nima Connect, Nima Wrapped, the tailor portal and seller try-on links on the roadmap?

8. **Kenya Data Protection Act:** consent for body photos, deletion, registration; rewrite the privacy policy and terms.
9. **iOS credit prices:** charge more on iPhone to cover Apple's cut, or absorb it?
10. **Ask Nima chat:** free on our VPS (slower, ≈ 5–10 words/s), or a cheap hosted model like Gemini Flash (fast, pennies per chat)?
11. **Google account:** can we reuse the old Gemini API key and billing account?

Source: `docs/PLAN.md` in the Nima AI project. Inputs: the Nima AI brief, the Canva design, audits of the previous code, and research on open-source models, licenses and Kenyan integrations. Figures marked "est." or "≈" are estimates, not measurements.